

Agentic AI in Practice

From Spec to Productive Workflow

Tobias Weiß · DevOps, University of Marburg

GöAID Session · Academic Cloud

Where I come from: an operation, not a lab

- openEduSuite: sovereign groupware suite for teaching, research, administration — on bare-metal Kubernetes.
- **AI-infused** means two things here: LLM services *in* the suite, and agents working *on* the operation.
- Production, not a demo: real mailboxes, real students, real outages.

This session is the story of how specs turned into productive workflows.

Roadmap — 30 minutes, three blocks

Block	Content	Time
1 · Foundations	What "agentic" concretely means: model + harness + loop	~8 min
2 · Method	Spec → Contract → Agent → Verify: the pipeline	~10 min
3 · Practice	Four cases from the openEduSuite operation	~12 min

Afterwards: 10 minutes of questions.

Agentic ≠ chat

- Chat: question → answer. Agent: **goal** → **plan** → **tools** → **result** → **feedback**.
- Three building blocks: model + tools (terminal, files, git, cluster) + loop (plan, act, verify).
- The difference does not live in the model — it lives in the **harness** around it.

"Agentic" is not a model feature, it is a working environment.

The harness determines what the model can achieve

- Same model, different frame: chat window vs. terminal agent with file, git and cluster access.
- Our harness: **pi** — terminal agent, MCP tools, skills, memory across sessions.
- **Plan mode**: read and plan first, write later — the agent shows the plan before it touches anything.

What the agent may do is written in the contract — not left to gut feeling.

pi in 60 seconds

```
curl -fsSL https://pi.dev/install.sh | sh # macOS / Linux  
npm install -g --ignore-scripts @earendil-works/pi-coding-agent
```

- **v1.0** (Oct 2026): fullscreen TUI · codemode –40% prompt tokens · image generation · hardened MCP OAuth.
- **Ctrl+L** — switch models mid-session · **Ctrl+P** cycles favorites · **Shift+Tab** cycles thinking level.
- Provider-agnostic: subscriptions, API keys or local models · `/hotkeys` lists every shortcut.

"Can the agent leak my keys?" — pi-sandbox

- An empirical study, not vibes: escape-attempt probes + a real exfiltration test across sandbox variants — evidence committed in the repo.
- The *recommended* Docker setup only stops **accidental** leaks — a malicious agent exfiltrates the one key you pass in.
- The *hardened* flags as documented don't run pi at all — `--read-only` breaks the session directory.
- What does work: seccomp + noexec (egress cut), non-root without network (local models), gVisor, secrets-broker — the real key never enters the sandbox.

Sandboxing is a measurement, not a checkbox. github.com/tobias-weiss-ai-xr/pi-sandbox

The pyramid: Spec · Contract · Test

Level	Says	Example 1: domain cutover	Example 2: SSO test suite
Spec	WHAT + acceptance	"Swap home.openedu → suite.graphwiz.ai; guards: 157 + 8 manifests untouched"	"All 15 SSO clients verified e2e: login, logout, scopes"
Contract	HOW the agent works	<code>AGENTS.md</code> : plan-mode before production; the script must be idempotent	<code>AGENTS.md</code> : every warning is a documented decision — "whatever" does not exist
Test	WHEN it is right	Re-run = no diff (idempotency proof)	<code>tests/sso</code> : 59 checks green → merge

Every level is text in the repo — work becomes repeatable instead of heroic.

From spec to change: OpenSpec

- A **change** = proposal + spec delta + tasks — versioned in the repo, readable by human *and* agent.
- The agent implements task by task; the human reviews and **archives**.
- Archived means: the spec on main is the new state — **documentation is the product**, not a by-product.

```
Spec (OpenSpec) → Contract (AGENTS.md) → Agent (pi)  
    → Verify (tests/CI) → Operate (ArgoCD)
```

OpenSpec × pi: the agent lives in the spec

- **pi-openspec** — `pi install npm:openspec-pi` — deep OpenSpec integration as a pi package.
- Native `openspec` tool: the agent queries `status`, `validate`, `show`, `archive` **during the task** — no copy-pasting specs.
- Auto-context: `openspec context` is injected into the system prompt at session start — invalidated when the spec tree changes.
- The workflow as slash commands: `/opsx-new`, `/opsx-apply`, `/opsx-verify`, `/opsx-archive` — plus matching skills.

The spec is not a document next to the agent — it is the agent's operating context.

The stage: openEduSuite on one slide

Component	Service	Component	Service
Identity	Keycloak (SSO)	Chat	Matrix/Synapse
Groupware	SOGGo	Projects	OpenProject
Files	OpenCloud	Video	Jitsi/Intercom
Wiki	XWiki	Portal	collab-dashboard

- GitOps: **ArgoCD** reconciles the cluster; bare metal (SCS), MariaDB/Galera, HAProxy.
- Agents work on this production system — hence: **spec before execution, test as acceptance.**

Case 1 · Domain migration of the whole suite — as a spec

- Starting point: cutover `home.openedu` → `suite.graphwiz.ai` — ingresses, certificates, OAuth redirects, mail domains across the whole stack.
- Human writes the **contract**: swap script idempotent, guard counters (157 + 8 manifests must *not* be touched), Keycloak redirects, Let's Encrypt wildcard.
- Agent implements `bootstrap-domain-swap.sh` — acceptance criteria green, only then execution.

| *A nerve-racking cutover becomes routine: you can simply run the script again.*

Case 2 · SSO mystery: white page after login

- Symptom: SOGo webmail shows a **white page** after SAML login — three root causes in a row.
- Agent in plan mode: hypotheses → probes (metadata dump, ACS URL, TLS hop at the ingress) → fix → test.
- Second case, same pattern: Stalwart XOAUTH2 broke on a dead OIDC issuer inside a directory object.

Debugging is the best agentic use case: symptom in, root-cause chain out — with a log you can reread.

Case 3 · "Fix-all": making drift visible

- Pattern: k8s directories with no ArgoCD app — tenant mail, LLM, XWiki; legacy from the imperative-apply era.
- Orders with acceptance criteria: every app under GitOps, every diff explained, nothing implicit.
- Side finding: `k8s/llm` — the model serving is part of the same suite.

Agents are great at the tedious work of whole directories — if the definition of done is in the contract.

Case 4 · An agent fleet with acceptance gates

- The SSO test suite (59 checks, 15 clients e2e) was built in batches: parallel workers, isolated git worktrees.
- Every task had exact acceptance criteria; merge only on green — otherwise the runner discards it.
- Fleet = spec at scale: write contracts once, distribute many small orders.

Not one super-agent — many small orders with tests. Scalable, cheap, auditable.

The tool behind the fleet: agentflow (af)

- Small Rust orchestrator: declarative `tasks.json` + `workers.json` → parallel LLM workers, each in an **isolated git worktree**.
- Merge only after the task's **acceptance gate** (shell command, exit 0) — retries get a fresh branch and the error history added to the prompt.
- Measured routing: every attempt leaves a receipt; free workers are picked by UCB1 (track record + exploration) — visible in `af cost`.
- Thin by design: it drives `git` and any OpenAI-compatible agent CLI (default: `pi`) — 71 tests, no network in CI.

Contracts once, many small orders — the runner enforces the gates. github.com/tobias-weiss-ai-xr/agentflow

What the operation learned

- **Tight specs over big contexts:** fewer tokens, same quality — the spec is the compressor.
- **Manage expectations:** agents do not deliver finished bots — they need contracts and tests.
- **Test first:** every incident becomes a test. Every warning in the suite is a documented decision.

| *Trust is good. Verification is cheaper.*

The pattern, applied to your project

1. **Contract first:** `AGENTS.md` in the repo — what the agent may do, where the facts live.
2. **Spec change before code:** small tasks with acceptance criteria — proposal → tasks → apply → archive.
3. **CI is the acceptance:** green = done. The agent may repeat itself; you review the result.
4. **Memory across sessions:** decisions land in the repo and in memory — not in the chat history.

Start with one boring, important task. Not with the experiment "an agent replaces me".

Resources

- **openEduSuite** — `openedusuite.graphwiz.ai` · portal: `openedu.graphwiz.ai`
- **pi** — `pi.dev` · install: `curl -fsSL https://pi.dev/install.sh | sh`
- **pi-openspec** — `github.com/tobias-weiss-ai-xr/pi-openspec` (`pi install npm:openspec-pi`)
- **pi-sandbox** — `github.com/tobias-weiss-ai-xr/pi-sandbox` · sandbox evidence + threat model
- **agentflow** — `github.com/tobias-weiss-ai-xr/agentflow` · parallel LLM tasks on worktrees, acceptance gates
- **Workshop repo** (3h deck + exercises) — GitHub/Codeberg: `IT4Science-Days-2026-agentic-ai-workshop`

Thank you!

Questions? — 10 minutes

Tobias Weiß · `tobias.weiss@uni-marburg.de`

GöAID · openEduSuite · University of Marburg